

SMART HOME AUTOMATION SYSTEM – HOME OS

A compact smart-home prototype that connects physical devices with a web dashboard.

The ESP32 Smart Home Automation System is a small IoT project that allows a user to control four room lights and monitor temperature and humidity from a web dashboard. The ESP32 is the main controller. It connects to Wi-Fi, reads the DHT11 sensor and controls four LEDs that represent lights in different rooms. The dashboard is designed as a single control screen. It shows live sensor readings, recent sensor graphs, room states, connection status and simple lighting scenes.

Goal of the Project

The main goal is to understand how a physical device can communicate with a web interface and how this idea can be used for home automation.

- Connect an ESP32 to a Wi-Fi network.
- Read temperature and humidity from a DHT11 sensor.
- Control physical outputs from a web page.
- Understand how HTTP sends control requests.
- Understand how WebSocket keeps live information updated.
- Keep the physical LED state and dashboard state synchronized

How the System Works

1. ESP32 starts: The ESP32 initializes the LEDs and DHT11 sensor.
2. Wi-Fi connection: The ESP32 joins the configured Wi-Fi network and receives an IP address.
3. Dashboard connection: A browser opens the dashboard hosted by the ESP32.
4. Control: The browser sends HTTP requests when a user changes a room light.
5. Sensor reading: The ESP32 reads temperature and humidity every few seconds.
6. Live update: Web Socket sends sensor values and room states to the dashboard.

HTTP Communication

HTTP is used mainly when the dashboard needs to ask the ESP32 to perform an action. A simple way to think about it is: the browser sends a request, and the ESP32 receives it and performs the requested action.

Example: Turning ON the Living Room When the user clicks the Living Room control, the browser sends a request similar to: **GET /yellow/on**

The ESP32 sees this request, understands that it means “Living Room ON”, and sets GPIO 16 HIGH. The LED then turns on.

Browser → HTTP request → ESP32 → GPIO changes → LED turns ON

WebSocket Communication

WebSocket is used for live updates. Instead of repeatedly asking for new information, the browser keeps a connection open with the ESP32. The ESP32 can then send new information whenever it has it.

In this project, WebSocket is used for temperature, humidity and room-state updates. The WebSocket connection uses port 81.

Example:

Sensor Data: The DHT11 may provide values such as: Temperature: 28.5 °C Humidity: 72.0 %
The ESP32 sends the values through the open WebSocket connection. The browser receives them and updates the numbers and graphs without refreshing the page.

Example:

Room State: The ESP32 can send a message such as: ROOMS: 1,0,0,0
This means Living Room = ON, Bedroom = OFF, Kitchen = OFF and Study Room = OFF. The dashboard uses this information to show the current state.

Data Flow:

DHT11 → ESP32 → WebSocket → Browser → Dashboard → Graph

User click → Browser → HTTP → ESP32 → GPIO → LED

After changing a light, the ESP32 sends the room states through WebSocket so the dashboard can show the actual current state.

What We Learned

The project gave us practical experience in connecting electronics, networking and web development.

- How an ESP32 connects to Wi-Fi and becomes reachable using its IP address.
- How a DHT11 sensor provides temperature and humidity readings.
- How GPIO pins can control physical outputs.
- How a web page can send a command to a physical device using HTTP.
- How WebSocket can keep live information flowing between the ESP32 and dashboard.
- How a dashboard can represent the real state of physical hardware.
- How different technologies can be combined to build a working IoT system.

Challenges Faced

- Establishing a stable Wi-Fi connection.
- Using the correct GPIO pins for each device.
- Making the dashboard respond correctly to user actions.
- Keeping the dashboard and physical LED states synchronized.
- Understanding when to use HTTP and when to use WebSocket.

Future Improvements

In the future, the project can be expanded by replacing the demo LEDs with relay modules to control real appliances, adding motion sensors for automatic lighting, and including fans and other devices. Sensor data can be stored for long-term monitoring, while user login and secure remote access can improve safety. Voice control and automated scenes can also make the system smarter and easier to use.

Conclusion

The ESP32 Smart Home Automation System demonstrates how a small controller can connect physical devices with a web-based interface. The project combines sensor monitoring, lighting control, Wi-Fi communication and a live dashboard. The key communication concepts are simple: HTTP is used to send control requests, while WebSocket is used to keep live information updated. Together, they allow the user interface and physical hardware to work as one system.

A small prototype of the basic idea behind IoT-based smart homes.